

8个乘积保存到向量寄存器`%ymm1`。我们看到，一条指令能够产生对多个数据值的计算，因此称为“SIMD”。

GCC 支持对 C 语言的扩展，能够让程序员在程序中使用向量操作，这些操作能够被编译成 AVX 的向量指令(以及基于早前的 SSE 指令的代码)。这种代码风格比直接用汇编语言写代码要好，因为 GCC 还可以为其他处理器上的向量指令产生代码。

使用 GCC 指令、循环展开和多个累积变量的组合，我们的合并函数能够达到下面的性能：

方法	整数				浮点数			
	int		long		long		int	
	+	*	+	*	+	*	+	*
标量 10×10	0.54	1.01	0.55	1.00	1.01	0.51	1.01	0.52
标量吞吐量界限	0.50	1.00	0.50	1.00	1.00	0.50	1.00	0.50
向量 8×8	0.05	0.24	0.13	1.51	0.12	0.08	0.25	0.16
向量吞吐量界限	0.06	0.12	0.12	—	0.12	0.06	0.25	0.12

上表中，第一组数字对应的是按照 `combine6` 的风格编写的传统标量代码，循环展开因子为 10，并维护 10 个累积变量。第二组数字对应的代码编写形式可以被 GCC 编译成 AVX 向量代码。除了使用向量操作外，这个版本也进行了循环展开，展开因子为 8，并维护 8 个不同的向量累积变量。我们给出了 32 位和 64 位数字的结果，因为向量指令在第一种情况中达到 8 路并行，而在第二种情况中只能达到 4 路并行。

可以看到，向量代码在 32 位的 4 种情况下几乎都获得了 8 倍的提升，对于 64 位来说，在其中的 3 种情况下获得了 4 倍的提升。只有长整数乘法代码在我们尝试将其表示为向量代码时性能不佳。AVX 指令集不包括 64 位整数的并行乘法指令，因此 GCC 无法为此种情况生成向量代码。使用向量指令对合并操作产生了新的吞吐量界限。与标量界限相比，32 位操作的新界限小了 8 倍，64 位操作的新界限小了 4 倍。我们的代码在几种数据类型和操作的组合上接近了这些界限。

5.10 优化合并代码的结果小结

我们极大化对向量元素加或者乘的函数性能的努力获得了成功。下表总结了对于标量代码所获得的结果，没有使用 AVX 向量指令提供的向量并行性：

函数	方法	整数		浮点数	
		+	*	+	*
<code>combine1</code>	抽象-O1	10.12	10.12	10.17	11.14
<code>combine6</code>	2×2 循环展开	0.81	1.51	1.51	2.51
	10×10 循环展开	0.55	1.00	1.01	0.52
延迟界限		1.00	3.00	3.00	5.00
吞吐量界限		0.50	1.00	1.00	0.50

使用多项优化技术，我们获得的 CPE 已经接近于 0.50 和 1.00 的吞吐量界限，只受限于功能单元的容量。与原始代码相比提升了 10~20 倍，且使用普通的 C 代码和标准编译器就获得了所有这些改进。重写代码利用较新的 SIMD 指令得到了将近 4 倍或 8 倍的性能提升。比如单精度乘法，CPE 从初值 11.14 降到了 0.06，整体性能提升超过 180 倍。这个例子说明现代处理器具有相当的计算能力，但是我们可能需要按非常程式化的方式来编写程序以便将这些能力诱发出来。

5.11 一些限制因素

我们已经看到在一个程序的数据流图表示中，关键路径指明了执行该程序所需时间的一个基本的下界。也就是说，如果程序中有某条数据相关链，这条链上的所有延迟之和等于 T ，那么这个程序至少需要 T 个周期才能执行完。

我们还看到功能单元的吞吐量界限也是程序执行时间的一个下界。也就是说，假设一个程序一共需要 N 个某种运算的计算，而微处理器只有 C 个能执行这个操作的功能单元，并且这些单元的发射时间为 I 。那么，这个程序的执行至少需要 $N \cdot I/C$ 个周期。

在本节中，我们会考虑其他一些制约程序在实际机器上性能的因素。

5.11.1 寄存器溢出

循环并行性的好处受汇编代码描述计算的能力限制。如果我们的并行度 p 超过了可用的寄存器数量，那么编译器会诉诸溢出(spilling)，将某些临时值存放到内存中，通常是在运行时堆栈上分配空间。举个例子，将 combine6 的多累积变量模式扩展到 $k=10$ 和 $k=20$ ，其结果的比较如下表所示：

函数	方法	整数		浮点数	
		+	*	+	*
combine6	10×10 循环展开	0.55	1.00	1.01	0.52
	20×20 循环展开	0.83	1.03	1.02	0.68
吞吐量界限		0.50	1.00	1.00	0.50

我们可以看到对这种循环展开程度的增加没有改善 CPE，有些甚至还变差了。现代 x86-64 处理器有 16 个寄存器，并可以使用 16 个 YMM 寄存器来保存浮点数。一旦循环变量的数量超过了可用寄存器的数量，程序就必须在栈上分配一些变量。

例如，下面的代码片段展示了在 10×10 循环展开的内循环中，累积变量 acc0 是如何更新的：

```
Updating of accumulator acc0 in 10 x 10 urolling
vmulsd  (%rdx), %xmm0, %xmm0    acc0 *= data[i]
```

我们看到该累积变量被保存在寄存器 %xmm0 中，因此程序可以简单地从内存中读取 data[i]，并与这个寄存器相乘。

与之相比，20×20 循环展开的相应部分非常不同：

```
Updating of accumulator acc0 in 20 x 20 unrolling
vmovsd  40(%rsp), %xmm0
vmulsd  (%rdx), %xmm0, %xmm0
vmovsd  %xmm0, 40(%rsp)
```

累积变量保存为栈上的一个局部变量，其位置距离栈指针偏移量为 40。程序必须从内存中读取两个数值：累积变量的值和 data[i] 的值，将两者相乘后，将结果保存回内存。

一旦编译器必须要诉诸寄存器溢出，那么维护多个累积变量的优势就很可能消失。幸运的是，x86-64 有足够多的寄存器，大多数循环在出现寄存器溢出之前就将达到吞吐量限制。